

---

# HelpDesk Pro

IT Support Ticketing & Analytics System

---

## PROJECT REPORT

**Vats Sanghvi**

Computer Applications Development

2026

### Tech Stack

Python · Django 5.1 · Django REST Framework · SQLite · Chart.js · Bootstrap 4

---

# Table of Contents

- 1. Executive Summary .....
- 2. Problem Statement.....
- 3. Project Objectives .....
- 4. System Architecture .....
- 4.1 Application Structure .....
  - 4.2 Technology Stack.....
- 5. Database Design.....
- 5.1 Models Overview .....
  - 5.2 New SLA Tracking Fields.....
  - 5.3 User Roles & Permissions.....
  - 5.4 Ticket Status Workflow .....
- 6. Features & Functionality .....
- 6.1 Analytics Dashboard.....
  - 6.2 Ticket Lifecycle Management .....
  - 6.3 REST API Endpoints .....
  - 6.4 Data Export Capabilities.....
  - 6.5 Security & Configuration.....
- 7. Sample Data — Manulife Use Case .....
- 8. Key Outcomes & Skills Demonstrated .....
- 8.1 Technical Outcomes.....
  - 8.2 Skills Demonstrated .....
- 9. Future Enhancements .....
- 10. Conclusion .....

*Note: Open this file in Microsoft Word, then press Ctrl + A followed by F9 to update all page numbers automatically.*

## 1. Executive Summary

HelpDesk Pro is a full-stack IT support ticketing and analytics system built with Python and Django, designed to streamline IT service management for enterprise environments. The system addresses a core challenge faced by IT support teams: the lack of a centralized, data-driven platform to track, manage, and resolve employee IT issues efficiently.

The application was designed and developed end-to-end — from database architecture and REST API design to a fully redesigned user interface with a real-time analytics dashboard. The project demonstrates practical application of software engineering, data management, role-based security, and business intelligence concepts directly relevant to Data Analyst, Business Analyst, and Support Analyst roles.

|              |                                                                           |
|--------------|---------------------------------------------------------------------------|
| Project Name | HelpDesk Pro — IT Ticketing & Analytics System                            |
| Developer    | Vats Sanghvi                                                              |
| Institution  | Computer Applications Development                                         |
| Tech Stack   | Python, Django 5.1, Django REST Framework, SQLite, Chart.js, Bootstrap 4  |
| API Layer    | Django REST Framework — 10+ endpoints with filtering, search & pagination |
| Database     | SQLite (development) / MySQL (production-ready)                           |
| Year         | 2026                                                                      |

## 2. Problem Statement

---

Organizations without a structured IT ticketing system face significant operational challenges that impact productivity and accountability:

- IT issues are reported informally via email or chat — no tracking, no accountability, no audit trail.
- No visibility into ticket status, priority, or resolution time for management or employees.
- Support teams cannot prioritize work effectively without SLA tracking or workload data.
- Leadership lacks data-driven insights to identify recurring issues or measure team performance.
- Manual reporting is time-consuming and error-prone — no structured way to export data for analysis.

HelpDesk Pro solves all of the above by providing a single centralized platform that manages the full IT ticket lifecycle from creation to resolution, with built-in real-time analytics and reporting capabilities.

## 3. Project Objectives

---

- Build a structured ticket management system with full lifecycle tracking across 7 status stages.
- Implement role-based access control ensuring each user type sees only relevant data and actions.
- Design and expose a REST API to serve the analytics dashboard and enable future integrations.
- Develop a real-time analytics dashboard with KPI cards and interactive Chart.js visualizations.
- Implement SLA deadline tracking to flag overdue tickets and measure team compliance.
- Enable CSV and Excel data exports to support analyst-level reporting and downstream analysis.
- Apply security best practices: environment-based config, no hardcoded credentials, complete audit logging.

## 4. System Architecture

### 4.1 Application Structure

The application follows Django's MTV (Model-Template-View) pattern with a clear separation between the web layer and the REST API layer:

| Layer          | Component           | Description                                                         |
|----------------|---------------------|---------------------------------------------------------------------|
| Project Config | ticketit/           | Django settings, URL routing, WSGI/ASGI configuration               |
| Auth App       | registration/       | Custom User model, login, user management views                     |
| Core App       | vats/               | Ticket, Category, Subcategory, Worknote models + all business logic |
| REST API       | vats/api_views.py   | DRF ViewSets, analytics endpoints, CSV & Excel export               |
| Templates      | templates/          | Django HTML templates — sidebar layout, dashboard, forms            |
| Static Assets  | static/css/main.css | Custom design system layered on top of Bootstrap 4                  |

### 4.2 Technology Stack

| Technology            | Version | Role & Justification                                               |
|-----------------------|---------|--------------------------------------------------------------------|
| Python                | 3.13    | Primary backend language — industry standard for data-heavy apps   |
| Django                | 5.1.4   | Web framework — ORM, built-in admin, auth, URL routing             |
| Django REST Framework | 3.15.2  | REST API — serializers, ViewSets, browsable API for testing        |
| django-filter         | 24.3    | Server-side filtering — filter tickets by any field combination    |
| django-environ        | 0.11.2  | Environment-based config — removes hardcoded secrets from codebase |
| openpyxl              | 3.1.5   | Excel export with styled header rows — key for analyst workflows   |
| SQLite / MySQL        | —       | SQLite for dev; production-ready for MySQL migration               |
| Chart.js              | 4.4.1   | Interactive bar and donut charts on the analytics dashboard        |
| Bootstrap 4           | 4.3.1   | Responsive UI grid system and base components                      |

## 5. Database Design

### 5.1 Models Overview

The system consists of 5 core models with clearly defined relationships:

| Model       | Key Fields                                                                                   | Relationships                                       |
|-------------|----------------------------------------------------------------------------------------------|-----------------------------------------------------|
| User        | email, first_name, last_name, phone_number, role, is_active                                  | Custom auth model — FK target for Ticket & Worknote |
| Category    | name                                                                                         | One-to-many with Subcategory and Ticket             |
| Subcategory | name, category (FK)                                                                          | Many-to-one with Category; FK on Ticket             |
| Ticket      | number, title, problem_descp, priority, status, due_by, resolved_at, created_by, assigned_to | Central model — connects all other entities         |
| Worknote    | comment, type, field_name, old_value, new_value, ticket (FK), commented_by (FK)              | Audit trail — one-to-many with Ticket               |

### 5.2 New Fields Added in v2 (SLA Tracking)

| Field       | Type                     | Purpose                                                                             |
|-------------|--------------------------|-------------------------------------------------------------------------------------|
| due_by      | DateTimeField (nullable) | Auto-calculated on creation based on priority: High = 4h, Moderate = 24h, Low = 72h |
| resolved_at | DateTimeField (nullable) | Auto-stamped when status changes to Completed, Cancelled, or Rejected               |

### 5.3 User Roles & Permissions

| Role    | Permissions                                                                                      |
|---------|--------------------------------------------------------------------------------------------------|
| Admin   | Full access — create/manage users, approve/reject tickets, manage categories, view all analytics |
| Manager | Handle assigned tickets — move through Scoping > In Progress > Completed, add work notes         |
| Viewer  | Raise new tickets, track own ticket status, add comments on own tickets                          |

### 5.4 Ticket Status Workflow

| Status      | Meaning                                   | Triggered By                 |
|-------------|-------------------------------------------|------------------------------|
| Pending     | Newly raised, awaiting admin review       | Automatic on ticket creation |
| Assigned    | Approved and assigned to a manager        | Admin approval action        |
| Scoping     | Manager is assessing the scope and impact | Manager action               |
| In Progress | Actively being worked on                  | Manager action               |
| Completed   | Issue resolved successfully               | Manager action               |
| Cancelled   | Ticket withdrawn by viewer or manager     | Viewer or Manager            |
| Rejected    | Ticket rejected by admin                  | Admin action                 |

## 6. Features & Functionality

### 6.1 Analytics Dashboard

The home page serves as a live analytics dashboard that fetches data from the REST API and renders interactive visualizations in real time:

- 4 KPI cards: Total Tickets, Open, Resolved, and Average Resolution Time (hours)
- Bar chart: Ticket volume by day for the past 7 days — broken down by Open, Resolved, and Pending
- Donut chart: Priority distribution across High, Moderate, and Low with percentage labels
- Category breakdown: Horizontal progress bars showing ticket volume per category
- Team workload section: Per-manager open vs. resolved counts (Admin view only)
- Recent tickets table: Latest 8 tickets with status pills, priority badges, and direct detail links

### 6.2 Ticket Lifecycle Management

- 7 distinct status stages with role-gated transitions enforced at both view and API level
- SLA tracking: due\_by auto-calculated on creation; overdue tickets flagged with a red SLA badge on the dashboard
- Complete audit trail: every status change and field update logged via the Worknote model with actor and timestamp
- Activity feed: ticket detail page shows a chronological timeline of all changes and comments
- WhatsApp notifications via Twilio API on ticket creation and each status change
- Email notifications via Gmail SMTP on ticket creation and status updates

### 6.3 REST API Endpoints

A full REST API layer was added in v2, powering the live dashboard and enabling future integrations:

| Endpoint                     | Method      | Description                                            |
|------------------------------|-------------|--------------------------------------------------------|
| api/v1/tickets/              | GET / POST  | List tickets (role-filtered) or create a new ticket    |
| api/v1/tickets/<id>/         | GET / PATCH | Full ticket detail or partial update                   |
| api/v1/tickets/export/csv/   | GET         | Download all filtered tickets as a CSV file            |
| api/v1/tickets/export/excel/ | GET         | Download all filtered tickets as an Excel (.xlsx) file |
| api/v1/analytics/dashboard/  | GET         | KPI cards + all chart data in a single API call        |

| Endpoint                              | Method | Description                                      |
|---------------------------------------|--------|--------------------------------------------------|
| api/v1/analytics/report/              | GET    | Configurable date-range analytics report         |
| api/v1/categories/                    | GET    | Category list with ticket counts                 |
| api/v1/categories/<id>/subcategories/ | GET    | Subcategory list for a specific category         |
| api/v1/users/managers/                | GET    | List of active managers for assignment dropdowns |
| api/v1/users/me/                      | GET    | Currently authenticated user's profile           |

## 6.4 Data Export Capabilities

- CSV and Excel exports available for Admin and Manager roles from the ticket list page
- Exported fields: Ticket #, Title, Category, Priority, Status, Created By, Assigned To, SLA Due By, Resolved At, Resolution Time (hrs), SLA Breached
- Excel export includes a styled header row with blue background and bold white text
- All active filters carry through to the export — users download exactly the data they filtered to

## 6.5 Security & Configuration

- All secrets (Django key, Gmail password, Twilio API keys) stored in a .env file — never committed to GitHub
- SLA hours per priority level are configurable via .env without any code changes required
- Role-based access enforced at both the Django view level (decorators) and REST API level (DRF permissions)
- Session-based authentication for the web UI; API ready for token auth for future mobile or third-party use

## 7. Sample Data — Manulife Use Case

The system was populated with realistic IT helpdesk data themed around Manulife Financial, a major Canadian financial services company, to demonstrate real-world enterprise applicability:

| Data Type     | Count | Details                                                                                        |
|---------------|-------|------------------------------------------------------------------------------------------------|
| Categories    | 5     | Hardware, Software & Applications, Network & Connectivity, Security & Access, Data & Reporting |
| Subcategories | 25    | 5 per category — e.g. Bloomberg Terminal, Power BI, VPN & Remote Access, MFA Setup             |
| Users         | 9     | 1 Admin (IT Head), 3 Managers (IT leads), 5 Viewers (Manulife employees)                       |
| Tickets       | 30    | Spread across all 7 statuses, 3 priorities, and all 5 categories                               |
| Worknotes     | 50+   | Comment entries and field-change audit entries on each ticket                                  |

Sample ticket scenarios include realistic Manulife IT cases: Bloomberg Terminal freezing during market hours (High priority, trading impact), SAP authorization error blocking month-end close (High, finance impact), Power BI dashboard showing stale data for executive meeting (High, leadership impact), and routine items like password resets, VPN configuration, and new user provisioning.

## 8. Key Outcomes & Skills Demonstrated

### 8.1 Technical Outcomes

- Delivered a production-ready Django 5.1 application with clean separation between web views and REST API layer.
- Designed a complete relational database schema with 5 models, proper FK relationships, and 2 new analytics-enabling fields added via Django migrations.
- Built 10+ REST API endpoints with role-based queryset filtering, full-text search, multi-parameter filtering, and pagination.
- Implemented automated SLA deadline tracking — `due_by` auto-calculated from priority on creation; `is_sla_breached` exposed as a computed model property.
- Developed CSV and Excel export pipelines with dynamic filter pass-through, enabling non-technical users to extract structured data independently.
- Resolved a security vulnerability by moving all hardcoded credentials (Gmail, Twilio, Django secret key) to environment variables.

### 8.2 Skills Demonstrated

| Skill Area          | How It Is Demonstrated in This Project                                             |
|---------------------|------------------------------------------------------------------------------------|
| Backend Development | Python 3.13, Django 5.1, custom User model, decorators, middleware                 |
| API Design          | RESTful API with DRF — ViewSets, serializers, DjangoFilterBackend, pagination      |
| Database Design     | Relational schema, Django ORM queries, migrations, computed model properties       |
| Data Analytics      | SLA metrics, resolution time calculation, aggregated KPIs, chart data preparation  |
| Data Export         | CSV & Excel generation with openpyxl — styled, filterable, download-ready          |
| UI/UX Design        | Custom CSS design system, sidebar layout, Chart.js visualizations, Bootstrap 4     |
| Security            | Environment config, role-based permissions, audit trail, .gitignore best practices |
| Version Control     | Git workflow, clean project structure, professional README, .env.example           |

## 9. Future Enhancements

---

- — **Connect the REST API directly to Power BI for advanced executive reporting and drill-down analytics.**Power BI Integration
- — **Weekly email summaries of SLA compliance rates and open ticket counts delivered to managers automatically.**Scheduled Email Digests
- — **Migrate from SQLite to MySQL and deploy to Railway or Render for a live publicly accessible demo link.**Cloud Deployment
- — **Allow users to attach screenshots and log files to tickets to speed up diagnosis and resolution.**File Attachments
- — **Enhance the sidebar layout for smaller screens to support field support teams on mobile devices.**Mobile-Responsive Layout

## 10. Conclusion

---

HelpDesk Pro demonstrates the practical application of full-stack web development, REST API design, database engineering, and data analytics in a real-world business context. The project evolved from a basic Django CRUD application into a professional-grade IT service management system with live analytics, SLA tracking, role-based access control, and structured data export capabilities.

The system is directly relevant to roles in Data Analysis, Business Analysis, and IT Support — showcasing the ability to design data pipelines, build analytics dashboards, and deliver user-facing applications that turn raw operational data into actionable business insights.

The Manulife-themed sample dataset grounds the project in a recognizable financial services context, demonstrating awareness of enterprise IT environments and the types of data challenges that large organizations face on a daily basis.

---

Vats Sanghvi — 2026  
[github.com/VatsSanghvi](https://github.com/VatsSanghvi) | [vatssanghvi7@gmail.com](mailto:vatssanghvi7@gmail.com)